

DSA PLACEMENT QUESTIONS

Core Concepts

1. What is Time Complexity?

- Measures execution time of algorithm
- Expressed using Big-O notation

2. What is Space Complexity?

- Memory used by algorithm

3. What is Big-O Notation?

- Describes worst-case complexity
- Example: $O(n)$, $O(\log n)$, $O(n^2)$

4. What is an Array?

- Collection of elements
- Stored in contiguous memory

5. What is a Linked List?

- Nodes connected via pointers
- Dynamic size

6. Difference: Array vs Linked List

- Array $\rightarrow$ fixed size, fast access
- Linked List $\rightarrow$ dynamic, slower access

Key Points:

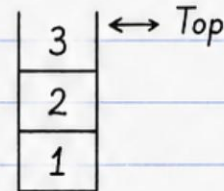
- Complexity questions are must for interviews
- Basics decide problem-solving ability

DSA PLACEMENT QUESTIONS

Data Structures

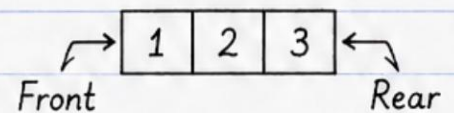
1. What is a Stack?

- Follows LIFO (Last In First Out)
- Operations: Push, Pop, Peek



2. What is a Queue?

- Follows FIFO (First In First Out)
- Operations: Enqueue, Dequeue, Front



3. What is a Deque (Double Ended Queue)?

- Insertions and deletions at both ends
- Types: Input Restricted, Output Restricted

4. What is a Hash Table?

- Stores key-value pairs
- Fast access using Hash Function

5. What is a Tree?

- Hierarchical data structure
- Types: Binary Tree, BST, AVL, Heaps

6. What is a Graph?

- Collection of vertices (nodes) and edges
- Types: Directed, Undirected, Weighted

7. What is a Heap?

- Complete Binary Tree
- Types: Min-Heap, Max-Heap

8. What is a Trie?

- Tree-like structure for strings
- Used in autocomplete, dictionary

Key Points:

- Know basic operations of each data structure
- Understand when to use which data struc'

DSA PLACEMENT QUESTIONS

Algorithms (Sorting + Searching)

1. What is Bubble Sort?

- Repeatedly swaps adjacent elements if they are in wrong order
- Time Complexity: $O(n^2)$ average and worst case
- Space Complexity: $O(1)$

Example:

Arr = [5, 1, 4, 2, 8]

Pass 1: [1, 4, 2, 5, 8]

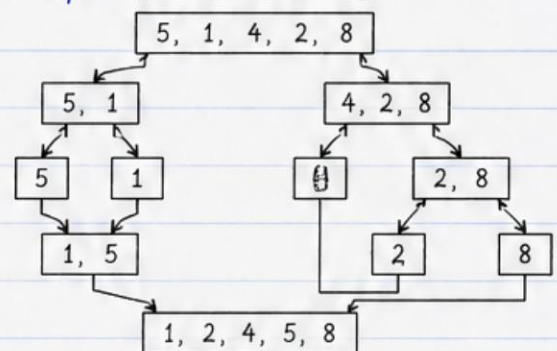
Pass 2: [1, 2, 4, 5, 8]

Pass 3: [1, 2, 4, 5, 8] (Sorted)

2. What is Merge Sort?

- Divide and Conquer algorithm
- Divides array into halves, sorts and merges
- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

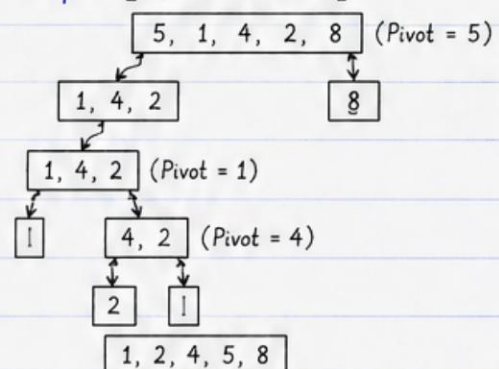
Example: [5, 1, 4, 2, 8]



3. What is Quick Sort?

- Divide and Conquer algorithm
- Uses pivot to partition array
- Time Complexity: $O(n \log n)$ average
 $O(n^2)$ worst case
- Space Complexity: $O(\log n)$
(recursion stack)

Example: [5, 1, 4, 2, 8]



4. What is Linear Search?

- Checks each element one by one
- Time Complexity: $O(n)$
- Best for unsorted or small data

Example: Arr = [10, 20, 30, 40, 50]

Key = 30

Check 10 → not found

Check 20 → not found

Check 30 → found at index 2

5. What is Binary Search?

- Works on sorted arrays
- Repeatedly divides array into half
- Time Complexity: $O(\log n)$
- Best for large sorted data

Example: Arr = [10, 20, 30, 40, 50, 60, 70]

Key = 40

Mid = 30 → key > 30, search right half

Mid = 50 → key < 50, search left half

Mid = 40 → found at index 3

Key Points:

- Choose algorithm based on constraints (time, space, input size)
- Sorting and Searching are most asked in interviews

DSA PLACEMENT QUESTIONS

Strings

1. What is a String?

- A sequence of characters
- Immutable in most programming languages (like Python)

Example:

`s = "hello"`

Characters: h, e, l, l, o

Length = 5

2. What is String Immutability?

- Strings cannot be changed after creation
- Any modification creates a new string

Example:

`s = "abc"`

`s[0] = 'z' × (Not allowed)`

`t = "z" + s[1:] ✓ (Creates new string)`

3. What is String Concatenation?

- Joining two or more strings

Example:

`s1 = "hello", s2 = "world"`

`s = s1 + " " + s2 → "hello world"`

4. What is Substring?

- A part of the original string
- Extracted using slicing

Example:

`s = "datastructure"`

`sub = s[4:9] → "struc"`

5. What is String Reversal?

- Reversing the characters of a string
- Techniques: slicing, two pointers

Example:

`s = "algorithm"`

`reverse = s[::-1] → "mhtiroglA"`

6. What is Palindrome?

- A string that reads the same forward and backward
- Ignore case and non-alphanumeric characters (in real scenarios)

Example:

`s = "madam" → Palindrome ✓`

`s = "A man, a plan, a canal, Panama"`

`→ Palindrome (ignoring spaces and punctuation)`

7. What is Anagram?

- Two strings are anagram if they contain the same characters with same frequency
- Order may be different

Example:

`s1 = "listen", s2 = "silent"`

`→ Anagram ✓`

`s3 = "python" → Not an anagram ×`

Key Points:

- Strings are widely used in interviews (parsing, pattern matching, hashing)
- Practice problems on substrings, anagrams, palindromes, and frequency count

DSA PLACEMENT QUESTIONS

Recursion

1. What is Recursion?

- A function calling itself to solve a smaller subproblem
- Base case is used to stop the recursion

Example: Factorial

```
def fact(n):
    if n == 0: return 1 # Base case
    return n * fact(n-1) # Recursive call
```

2. What are the Components of Recursion?

- Base Case (stops recursion)
- Recursive Case (breaks into smaller problems)
- Progress Towards Base Case

3. What is the Time Complexity of Recursion?

- Depends on number of recursive calls
- Example: Factorial $\rightarrow O(n)$
Fibonacci $\rightarrow O(2^n)$ naive

Example: Fibonacci (Naive)

```
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

4. What is Tail Recursion?

- Recursive call is the last operation
- Can be optimized by some compilers (not in Python)

Example: Tail Recursion (Factorial)

```
def fact_tail(n, acc=1):
    if n == 0: return acc
    return fact_tail(n-1, n*acc)
```

5. What is the Difference: Iteration vs Recursion?

- Iteration uses loops, recursion uses function calls
- Recursion uses more memory (call stack)
- Iteration is generally faster

Iteration	Recursion
Uses loops	Uses function calls
Less memory	More memory (stack)
Faster	Slower

6. What Causes Stack Overflow in Recursion?

- Too many recursive calls without reaching base case
- Stack memory limit exceeded

Common Reasons:

- Missing or incorrect base case
- Infinite recursive calls

7. How to Optimize Recursion?

- Use Memoization (Top-Down DP)
- Use Iteration (Bottom-Up DP)

Example: Fibonacci (Memoized)

```
from functools import lru_cache
@lru_cache(None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Key Points:

- Master recursion: tree problems, backtracking, DP
- Always think of base case and how to reduce the problem

DSA PLACEMENT QUESTIONS

Backtracking

1. What is Backtracking?

- An algorithmic technique for solving problems by trying possible solutions and abandoning those that fail.
- It builds solution step by step and backtracks when a choice leads to a dead end.

Example:

Find all subsets of [1, 2, 3]

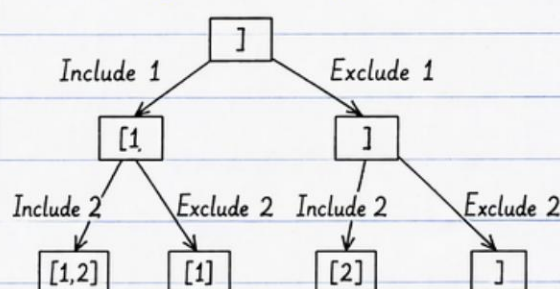
Output:

[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]

2. How Does Backtracking Work?

- Choose: Pick a choice
- Explore: Move forward
- Unchoose (Backtrack): If it doesn't work, go back and try another choice

Backtracking Tree (Example: Subsets of [1,2])



3. Characteristics of Backtracking

- Tries all possible solutions
- Abandons invalid or non-promising paths
- Uses recursion (implicit stack)
- Used for optimization and constraint problems

When to Use Backtracking?

- When problem has multiple solutions
- When you need to explore all possibilities
- When you can discard partial solutions early (pruning)

4. Common Backtracking Problems

- N-Queens Problem
- Sudoku Solver
- Rat in a Maze
- Word Search
- Generate All Subsets / Permutations
- Combination Sum
- Palindrome Partitioning

Example: N-Queens (N = 4)

Q	-	-	-
-	-	Q	-
-	Q	-	-
-	-	-	Q

Place N queens on N×N board such that no two queens attack each other.

5. Backtracking Template (Pseudocode)

```

def backtrack(choices):
    if is_solution():
        save_solution()
        return
    for choice in choices:
        if is_valid(choice):
            make_choice(choice)
            backtrack(choices)
            undo_choice(choice) # Backtrack
  
```

Explanation:

1. Check if current state is a solution.
2. Loop through all choices.
3. If choice is valid, make the choice and move forward.
4. After exploring, undo the choice (backtrack) and try next option.

Key Points:

- Backtracking is brute force with intelligence (pruning).
- Always try to reduce search space using constraints.
- Important in coding interviews and competitive programming.

DSA PLACEMENT QUESTIONS

Dynamic Programming (DP)

1. What is Dynamic Programming?

- An optimization technique for solving problems that have overlapping subproblems and optimal substructure.
- It solves each subproblem once and stores the result to avoid redundant work.

2. When to Use DP?

- Overlapping Subproblems
- Optimal Substructure
- Can be broken down into smaller states

3. Two Approaches in DP

- Top-Down (Memoization)
 - Uses recursion + cache (dictionary/array)
 - Easy to write, may have recursion overhead
- Bottom-Up (Tabulation)
 - Uses iteration + table
 - More efficient, avoids recursion stack

4. Memoization vs Tabulation

Aspect	Memoization (Top-Down)	Tabulation (Bottom-Up)
Approach	Recursion + Cache	Iteration + Table
Implementation	Easy	Slightly complex
Space	$O(n)$ + Recursion stack	$O(n)$
Time	$O(n)$	$O(n)$
Best Use	When not all states are visited	When all states are needed
Stack Overflow	Possible	Not Possible

5. Common DP Problems

- Fibonacci Series
- 0/1 Knapsack
- Longest Common Subsequence (LCS)
- Longest Increasing Subsequence (LIS)
- Matrix Chain Multiplication
- Coin Change
- Edit Distance

Key Points:

- Identify states and define $dp[state]$ meaning
- Write base cases carefully
- Choose Memoization or Tabulation based on problem
- DP is all about breaking problem into smaller overlapping subproblems

6. Example 1: Fibonacci Series

Find nth Fibonacci number.

Top-Down (Memoization)

```
def fib(n, dp):
    if n <= 1: return n
    if dp[n] != -1: return dp[n]
    dp[n] = fib(n-1, dp) + fib(n-2, dp)
    return dp[n]
```

Bottom-Up (Tabulation)

```
def fib(n):
    dp = [0]*(n+1)
    dp[0] = 0, dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

7. Example 2: 0/1 Knapsack

Given weights[], values[] and capacity W, find max value we can put in the bag.

Bottom-Up (Tabulation)

```
dp[i][w] = max value using first i items and capacity w
for i in range(n+1):
    for w in range(W+1):
        if i == 0 or w == 0:
            dp[i][w] = 0
        elif weights[i-1] <= w:
            dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]],
                           dp[i-1][w])
        else:
            dp[i][w] = dp[i-1][w]
Answer = dp[n][W]
```

8. Example 3: Longest Common Subsequence (LCS)

Given two strings s1 and s2, find length of LCS.

Bottom-Up (Tabulation)

```
dp[i][j] = LCS length of s1[0..i-1] and s2[0..j-1]
for i in range(n+1):
    for j in range(m+1):
        if i == 0 or j == 0:
            dp[i][j] = 0
        elif s1[i-1] == s2[j-1]:
            dp[i][j] = 1 + dp[i-1][j-1]
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
Answer = dp[n][m]
```

9. DP Problem-Solving Steps

1. Understand the problem and write down states
2. Identify base cases
3. Write recurrence relation
4. Choose Memoization or Tabulation
5. Solve and optimize space if possible

DSA PLACEMENT QUESTIONS

Important Data Structures

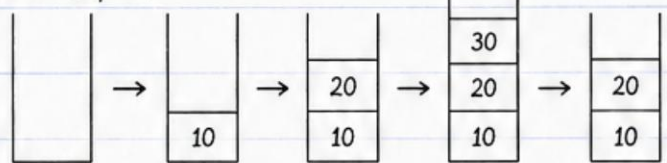
1. Stack (LIFO)

- Last In, First Out
- Operations: Push, Pop, Peek, isEmpty
- Used in: Recursion, Backtracking, Expression Evaluation, Undo/Redo

Example: Stack

Push 10, Push 20, Push 30, Pop()

Stack (top →)



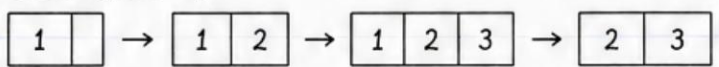
2. Queue (FIFO)

- First In, First Out
- Operations: Enqueue, Dequeue, Front, isEmpty
- Used in: Scheduling, BFS, Print Queue

Example: Queue

Enqueue 1, Enqueue 2, Enqueue 3, Dequeue()

Queue (front →)



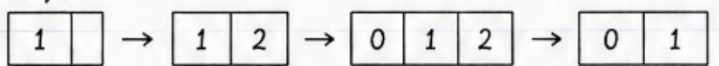
3. Deque (Double Ended Queue)

- Insert and delete at both ends
- Operations: InsertFront, InsertRear, DeleteFront, DeleteRear
- Used in: Sliding Window Maximum, Palindrome Check

Example: Deque

InsertRear 1, InsertRear 2, InsertFront 0, DeleteRear()

Deque



4. Hash Table (Dictionary / Map)

- Stores key-value pairs
- Operations: Insert, Search, Delete
- Average Time Complexity: $O(1)$
- Used in: Frequency Count, Caching, Maps

Example: Hash Table

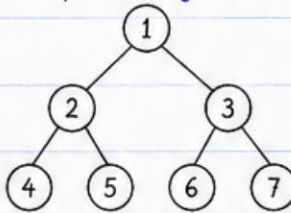
Insert ("a", 1), ("b", 2), ("c", 3)

Index	0	1	2	3	4	5	6	7
Value	-	1	-	2	-	3	-	-
		↑ "a"		↑ "b"		↑ "c"		

5. Binary Tree

- Each node has at most two children
- Types: Full, Complete, Perfect, BST
- Traversals: Inorder, Preorder, Postorder, Level Order
- Used in: Hierarchical Data, Searching

Example: Binary Tree



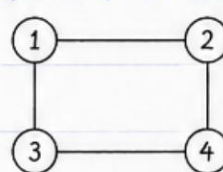
Traversals

Inorder : 4 2 5 1 6 3 7
 Preorder : 1 2 4 5 3 6 7
 Postorder : 4 5 2 6 7 3 1
 Level Order : 1 2 3 4 5 6 7

6. Graph

- Collection of vertices (nodes) and edges
- Types: Directed/Undirected, Weighted/Unweighted
- Representations: Adjacency List, Adjacency Matrix
- Used in: Networks, Maps, Dependencies

Example: Graph (Undirected)



Adjacency List

1 → [2, 3]
 2 → [1, 4]
 3 → [1, 4]
 4 → [2, 3]

Key Points:

- Know operations, time complexities and use-cases of each data structure.
- Choosing the right data structure improves both time and space efficiency.
- Practice implementing these from scratch.

DSA PLACEMENT QUESTIONS

Complexity Analysis

1. Asymptotic Notations

- Describe growth rate of functions
- Ignore constants and lower order terms
- Main notations: O (Big- O), Ω (Omega), Θ (Theta), o (little- o)

Notation	Name	Definition	Example
$O(f(n))$	Big- O	Upper bound	$3n^2 + 2n + 1 = O(n^2)$
$\Omega(f(n))$	Omega	Lower bound	$3n^2 + 2n + 1 = \Omega(n^2)$
$\Theta(f(n))$	Theta	Tight bound	$3n^2 + 2n + 1 = \Theta(n^2)$
$o(f(n))$	little- o	Strict upper bound	$n = o(n \log n)$
$\omega(f(n))$	little-omega	Strict lower bound	$n \log n = \omega(n)$

2. Common Complexity Orders

- Best: $O(1)$
- Average: $O(\log n)$, $O(n)$, $O(n \log n)$
- Worst: $O(n^2)$, $O(2^n)$, $O(n!)$

Order	Name	Example
$O(1)$	Constant	Access array element
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort
$O(n^2)$	Quadratic	Bubble Sort, Selection Sort
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Travelling Salesman (Brute Force)

3. Time vs Space Complexity

- Time Complexity: How time grows with input size
- Space Complexity: How memory grows with input size
- Optimize based on problem constraints

Time Complexity	Space Complexity
Focus on speed	Focus on memory
Depends on operations	Depends on extra space used
Example: $O(n \log n)$	Example: $O(n)$

4. How to Analyze Complexity?

- Identify basic operation
- Count how many times it executes
- Express in terms of n (input size)
- Simplify using asymptotic notation

Example:

```
for i = 1 to n:
    for j = 1 to n:
        print(i, j)
```

- Basic operation: `print()`
- Executes $n * n$ times
- Time Complexity: $O(n^2)$

5. Common Patterns and Their Complexities

- Single Loop: $O(n)$
- Nested Loop: $O(n^2)$
- Three Nested Loops: $O(n^3)$
- Divide and Conquer (balanced): $O(n \log n)$
- Recursion (exponential branching): $O(2^n)$

Pattern	Time Complexity
Single loop	$O(n)$
Nested loop (2 loops)	$O(n^2)$
Nested loop (3 loops)	$O(n^3)$
Binary Search	$O(\log n)$
Merge Sort	$O(n \log n)$
Fibonacci (naive recursion)	$O(2^n)$

6. Rules to Remember

- Drop constants: $O(5n) = O(n)$
- Drop lower order terms: $O(n^2 + n) = O(n^2)$
- Keep highest order term
- Always mention complexity in terms of n

Example

- $O(4n^2 + 10n + 3) = O(n^2)$
- $O(7n \log n + 5n) = O(n \log n)$
- $O(100) = O(1)$

Key Points:

- Complexity analysis helps in writing efficient code.
- Always think about worst-case unless mentioned otherwise.
- Interviews focus more on how you think, not just the answer.

DSA PLACEMENT QUESTIONS

Recursion

1. What is Recursion?

- A function calling itself to solve smaller instances of the same problem.
- Base case stops the recursion.

2. Structure of Recursive Function

- Base Case
- Recursive Call
- Processing (if any)

3. Example: Factorial

```
int fact(int n) {
    if (n == 0 || n == 1) return 1; // Base case
    return n * fact(n - 1); // Recursive call
}
```

4. Example: Fibonacci

```
int fib(int n) {
    if (n <= 1) return n; // Base case
    return fib(n - 1) + fib(n - 2); // Recursive call
}
```

5. Common Uses of Recursion

- Tree / Graph Traversal
- Divide and Conquer
- Backtracking
- Mathematical Problems (Fibonacci, Factorial, etc.)

6. Pros and Cons of Recursion

Pros	Cons
- Code is shorter and cleaner - Easy to understand - Matches natural problem structure	- Extra memory for call stack - Slower for large inputs - Risk of Stack Overflow

Recursion Call Stack (Factorial of 4)

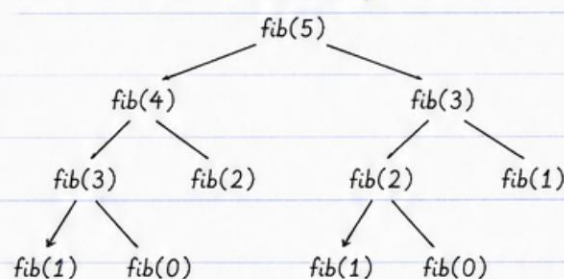
```
fact(4)
↓
fact(3)
↓
fact(2)
↓
fact(1)
↓
fact(0)
return 1
```

Unwinding (Returns)

```
fact(1) returns 1
fact(2) returns 2 * 1 = 2
fact(3) returns 3 * 2 = 6
fact(4) returns 4 * 6 = 24

Answer = 24
```

Recursion Tree (Fibonacci for n = 5)



Time Complexity of Fibonacci (Naive Recursion)

$$T(n) = T(n-1) + T(n-2) + O(1)$$

Time Complexity: $O(2^n)$

(Can be optimized using Memoization / DP)

Tips to Solve Recursive Problems

1. Identify base case.
2. Make the problem smaller.
3. Assume the recursive call works.
4. Combine the results.
5. Draw recursion tree for clarity.

Backtracking

1. What is Backtracking?

- An algorithmic technique to build solutions step by step and remove choices that don't lead to a solution.
- It is like "Try and Undo".

2. When to Use Backtracking?

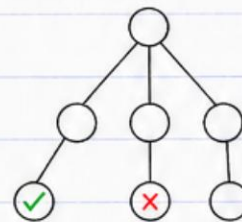
- When problem has multiple solutions
- When you need all possible solutions
- When you can abandon partial solutions early

3. Common Backtracking Problems

- N-Queens Problem
- Rat in a Maze
- Subset Sum
- Word Search
- Sudoku Solver
- Permutations / Combinations

Backtracking Process

1. Make a choice
2. Explore further
3. If valid solution found → store/print
4. If not, undo the choice (backtrack)
5. Try next choice



Backtracking Template (Pseudocode)

```
void solve(choices) {
    if (isSolution(choices)) {
        saveSolution(choices);
        return;
    }
    for (each choice in availableChoices) {
        makeChoice(choice); // Explore
        solve(choices);      // Recurse
        undoChoice(choice);  // Backtrack
    }
}
```

Key Points:

- Recursion solves by solving smaller subproblems.
- Backtracking explores choices and backtracks on invalid paths.
- Master recursion first, backtracking becomes easier!

DSA PLACEMENT QUESTIONS

Backtracking Examples

1. N-Queens Problem

- Place N queens on an $N \times N$ chessboard so that no two queens attack each other.
- A queen attacks in same row, column and diagonals.
- Classic Backtracking problem.

Example Solution for $N = 4$

0		Q		
1				Q
2	Q			
3			Q	
	0	1	2	3

Approach (Pseudocode)

```
bool isSafe(row, col):
    Check column and both diagonals
    for previous rows

void solve(row):
    if row == N:
        store solution
        return
    for col in 0 to N-1:
        if isSafe(row, col):
            place queen
            solve(row + 1)
            remove queen (backtrack)
```

Tips

- Place one queen in each row
- Check only previous rows
- Backtrack when no position is possible

2. Sudoku Solver

- Fill a 9×9 grid so that each row, each column and each 3×3 subgrid contains digits 1 to 9 exactly once.
- Backtracking with constraint checking.

Example (Partially Filled)

0	5	3	.	.	7	.	.	.
1	6	.	.	1	5	.	.	.
2	.	9	8	.	.	.	6	.
3	8	.	.	.	6	.	.	3
4	4	.	.	8	.	3	.	1
5	7	.	.	.	2	.	.	6
6	.	6	.	.	.	2	8	.
7	.	.	.	4	1	9	.	5
8	.	.	.	.	8	.	7	9

Approach (Pseudocode)

1. Find an empty cell (r, c)
2. Try digits 1 to 9
 - If safe (not in row, column and 3×3 box), place it
 - Recur for next cell
3. If no digit works, backtrack (remove and try next)
4. If no empty cell left, solution found

Safety Check

A number is safe if:

- Not present in same row
- Not present in same column
- Not present in same 3×3 box

★ Use helper functions: `isSafe(r, c, num)` and `findEmpty()`

3. Rat in a Maze

- A rat starts from top-left $(0,0)$ and needs to reach bottom-right $(N-1, N-1)$.
- 1 = open cell, 0 = blocked cell
- Find any path (or all paths).

Example Maze (4×4)

0	1	0	0	0
1	1	1	0	1
2	0	1	1	1
3	0	0	1	1
	0	1	2	3

One Possible Path

1	→	0	0	0
↓		1	0	1
↓		0	→	1
↓		0	0	1

Approach (Pseudocode)

```
solve(r, c):
    if out of bounds or blocked → return false
    if (r, c) is destination → return true
    mark (r, c) as visited
    for each direction (D, L, R, U):
        if solve(new_r, new_c) == true:
            return true
    unmark (r, c) // Backtrack
    return false
```

Directions

- D → $(r+1, c)$
- L → $(r, c-1)$
- R → $(r, c+1)$
- U → $(r-1, c)$

★ You can modify to print all paths by storing the path string and exploring all options.

General Backtracking Template

```
// choices[] : all possible choices
// isValid(choice) : check if choice is valid
// makeChoice(choice) : do the choice
// unmakeChoice(choice) : undo the choice

void backtrack(int index) {
    if (isSolution(index)) {
        storeSolution();
        return;
    }
    for (each choice in choices[index]) {
        if (isValid(choice)) {
            makeChoice(choice);
            backtrack(index + 1);
            unmakeChoice(choice); // Backtrack
        }
    }
}
```

Backtracking vs Other Techniques

Technique	Best For	Explores	Backtracks
Backtracking	All possible solutions / Constraint problems	Exponential	Yes
Recursion	Divide into similar subproblems	Depends	Sometimes
Greedy	Local optimum (e.g., Activity Selection)	Polynomial	No
DP	Overlapping subproblems with optimal substructure	Polynomial	No (uses memo/table)

Key Takeaways

- Backtracking is about exploring choices step by step.
- Make a choice → Explore → If it doesn't work → Undo and try next.
- Used in: Combinatorial problems, puzzles, permutations, subset problems, ...
- Practice more problems to recognize patterns!

DSA PLACEMENT QUESTIONS

Important Patterns (Must Know for Interviews)

Many DSA problems follow certain patterns. Recognizing the pattern helps you solve them faster.

1. Sliding Window

Idea: Maintain a window (subarray / substring) and slide it to explore solutions efficiently.

When to use?

- Subarray / substring problems
- "At most", "At least", "Exactly" K problems
- Longest / Shortest subarray with a condition
- Fixed size or variable size window

Template (Variable Size Window)

```
l = 0
for r in range(n):
    add a[r] to window
    while window is not valid:
        remove a[l] from window
        l += 1
    update answer
```

Example: Longest Subarray with Sum $\leq K$

arr = [1, 2, 1, 0, 1, 1, 0], K = 3

i : 0 1 2 3 4 5 6
arr: [1 2 1 0 1 1 0]

r → [1 2 1 0 1 1 0]
↑
1 Sum = 4 (> 3) → shrink

r → [2 1 0 1 1 0]
↑
1 Sum = 3 (≤ 3) → update ans = 3

r → [1 2 1 0 1 1 0]
↑
1 Sum = 3 (≤ 3) → update ans = 4

r → [1 2 1 0 1 1 0]
↑
1 Sum = 3 (≤ 3) → update ans = 4 (max)

Common Problems

- Max Sum Subarray of size K
- Longest Substring without repeating characters
- Longest Subarray with Sum $\leq K$
- Minimum Window Substring
- Fruits Into Baskets
- Count of Anagrams in a String

Time Complexity
 $O(n)$

Space Complexity
 $O(1) / O(n)$

2. Two Pointer

Idea: Use two pointers to solve the problem efficiently (either from same side or opposite sides).

When to use?

- Sorted arrays / lists
- Pair / triplet sum problems
- Remove duplicates
- Partition problems
- Palindrome checking

Template (Opposite Direction)

```
l = 0, r = n - 1
while l < r:
    if condition_met:
        update answer
        l += 1 or r -= 1
    elif need_smaller:
        r -= 1
    else:
        l += 1
```

Example: Two Sum in Sorted Array

arr = [1, 2, 4, 7, 11, 15], target = 9

l = 0 r = 5

[1 2 4 7 11 15]
1 + 15 = 16 (> 9) → r--

l = 0 r = 4
[1 2 4 7 11 15]
1 + 11 = 12 (> 9) → r--

l = 0 r = 3
[1 2 4 7 11 15]
1 + 7 = 8 (< 9) → l++

l = 1 r = 3
[2 2 7 13]
2 + 7 = 9 → Found!

Common Problems

- Two Sum II
- 3Sum / 4Sum
- Remove Duplicates from Sorted Array
- Valid Palindrome
- Container With Most Water
- Trapping Rain Water

Time Complexity
 $O(n)$

Space Complexity
 $O(1)$

3. Prefix Sum

Idea: Precompute prefix sums to answer range queries or subarray problems in $O(1)$ or $O(n)$.

When to use?

- Range sum queries
- Subarray sum equals K
- Count subarrays with given sum
- 2D range sum queries

Prefix Sum (1D)

```
prefix[0] = 0
for i in range(1, n+1):
    prefix[i] = prefix[i-1] + arr[i-1]
# Sum of arr[l...r] = prefix[r+1] - prefix[l]
```

Example: Subarray Sum Equals K

arr = [3, 4, 7, 2, -3, 1, 4, 2], K = 7

Prefix Sum Array:

Index : 0 1 2 3 4 5 6 7 8
arr : - 3 7 7 2 -3 1 4 2
prefix : 0 3 7 14 16 13 14 18 20

We need $\text{prefix}[j] - \text{prefix}[i] = K$

⇒ $\text{prefix}[i] = \text{prefix}[j] - K$

Use a hashmap to store prefix sum frequencies.

Count = 4 (There are 4 subarrays with sum = 7)

Common Problems

- Range Sum Query
- Subarray Sum Equals K
- Continuous Subarray Sum
- Count of Subarrays with Given Sum
- 2D Range Sum Query

Time Complexity
 $O(n)$

Space Complexity
 $O(n)$

How to Identify Patterns?

Subarray / Substring with condition	→ Sliding Window
Pair / Triplet in Sorted Array	→ Two Pointer
Range queries / Subarray sum	→ Prefix Sum
Combinations / Permutations / N-Queens	→ Backtracking
Overlapping Subproblems + Optimal	→ Dynamic Programming

Tips

- ✓ Practice identifying patterns in problems.
- ✓ Solve similar problems to master the pattern.
- ✓ Write dry runs to understand better.
- ✓ Start simple, then

★ Key Takeaway

Master these patterns and you will solve 80% of interview

DSA PLACEMENT QUESTIONS

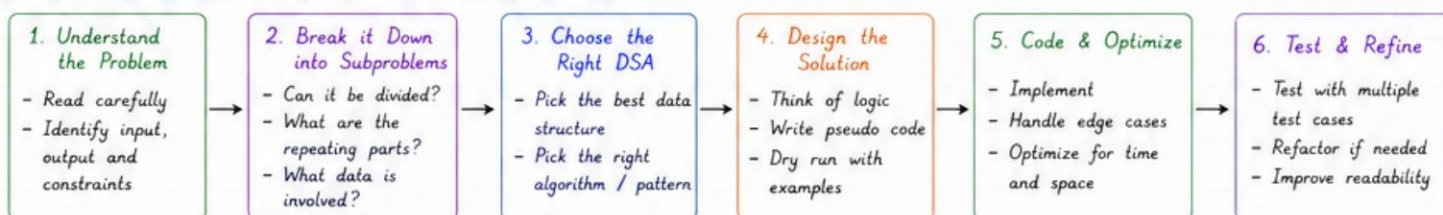
Real Life Problem Solving with DSA

Many real world problems can be efficiently solved using the right data structure and algorithm.

Below are common real life scenarios, the DSA concepts used and example problems.

Real Life Scenario	DSA Concepts Used	Why It Works	Examples
 1. E-Commerce Search products, filter, sort, recommendations	- Hashing (Search) - Sorting - Trie (Search Suggestions) - Heap / Priority Queue (Top K products)	Fast search, sorting for better results, suggestions based on prefix and popularity.	- Search autocomplete - Top K popular products - Filter products by price rating
 2. Navigation / Maps Find shortest path, traffic updates, nearest places	- Graph (Shortest Path) - BFS / Dijkstra - Heap (Dijkstra) - Hashing (Visited)	Graphs model roads and connections. Shortest path algorithms find optimal routes.	- Shortest path in maps - Nearest hospital / ATM - Real time traffic updates
 3. Social Media News feed, friend suggestions, count likes/comments	- Graph (Connections) - Heap (Top Feed) - Hashing (User Data) - Queue (Messaging)	Graphs for relationships, heaps for showing most relevant, queues for handling messages.	- News feed ranking - Friend suggestions - Real time chat message queue
 4. Banking / Finance Transactions, fraud detection, balance updates	- Hashing (Transactions) - Tree / BST (Range queries) - Sliding Window (Fraud Detection) - Stack (Expression Eval)	Fast lookups, range queries, pattern detection in time series, and safe expression evaluation.	- Detect unusual transactions - Account balance in range - Evaluate financial expressions
 5. Scheduling / Booking Book tickets, manage events, avoid overlaps	- Interval Scheduling - Priority Queue (Rooms) - Sorting - Sweep Line Algorithm	Efficiently manage time slots, avoid conflicts and maximize resource usage.	- Meeting room booking - Movie ticket booking - Event calendar management
 6. Cloud / File Systems Store and retrieve files, sync, version control	- Hashing (File Indexing) - Trie (File Paths) - Segment Tree / BIT (Range Updates) - Graph (Dependencies)	Fast file lookup, hierarchical structure, version control and dependency resolution.	- File search by name - Sync folder structure - Dependency management

Step-by-Step Approach to Solve Real Life Problems



Examples: Real Life Problems Mapped to DSA

Problem	Mapped DSA / Approach	Why?
Find nearest gas station on highway	Binary Search on sorted stations	Fast search in sorted data
Design parking lot	HashMap + Set	Track slots and fast availability check
LRU Cache	HashMap + Doubly Linked List	$O(1)$ get and put operations
File system structure	Trie / Tree	Hierarchical data representation
Top trending topics	HashMap + Heap (Top K)	Count frequency and find top K
Detect cycle in dependency	Graph + DFS	Find cycles in directed graph
Stock price span	Stack	Efficiently find previous greater element

General Tips

- ✓ Always relate problems to real life
- ✓ Start with brute force, then optimize
- ✓ Choose simple & best possible approach
- ✓ Draw diagrams for better clarity
- ✓ Think about edge cases
- ✓ Practice regularly on diverse problems

How to Practice Effectively?

- ★ Solve problems from platforms (LeetCode, GFG, Codeforces)
- ★ Try real world based problem lists
- ★ Build small projects (e.g., URL Shortener, Chat App, Task Scheduler)
- ★ Analyze and learn from editorials
- ★ Discuss approaches with peers

Mindset for Interviews

- 💡 Clarify the problem first
- 💡 Think out loud
- 💡 Explain your approach
- 💡 Write clean and modular code
- 💡 Test with examples
- 💡 Keep learning and stay

★ Remember:

DSA is not just about coding, it is about solving real problems efficiently and elegantly.

DSA PLACEMENT QUESTIONS

Real Life Problem Solving with DSA (Continued)

More real world scenarios and how DSA helps to solve them efficiently.

Real Life Scenario	DSA Concepts Used	Why It Works	Examples
 7. Logistics / Delivery Optimize delivery routes, track shipments	- Graph (Shortest Path) - Priority Queue - Hashing (Tracking) - Greedy (Nearest First)	Find shortest/fastest routes, prioritize deliveries and track status efficiently.	- Route optimization - Package tracking system - Delivery scheduling
 8. Health Care Manage patient records, appointment, analytics	- Hashing (Patient ID) - Heap (Priority Queue) - Tree (Hierarchical Data) - Sorting / Searching	Fast access to records, priority management and structured data handling.	- Appointment scheduling - Patient history lookup - Critical patient alerts
 9. Cyber Security Detect attacks, monitor logs, prevent threats	- Hashing (Signature Matching) - Trie (Pattern Matching) - Graph (Network Flow) - Sliding Window (Logs)	Pattern detection, anomaly identification and real time monitoring.	- Intrusion detection - Malware signature match - Log analysis
 10. Stock Market Analyze trends, predict, buy/sell decision	- Stack (Next Greater/Smaller) - Sliding Window (Moving Avg) - Segment Tree / BIT - Heap (Top K Stocks)	Efficient range analysis, trend detection and quick real time insights.	- Moving average calculation - Top gaining/losing stocks - Range query on prices
 11. Online Education Course recommendation, progress tracking, quizzes	- Graph (Recommendations) - Hashing (User Data) - Queue (Submission Order) - Sorting (Leaderboards)	Recommendations via graphs, ordered processing and performance tracking.	- Course recommendation - Quiz submission system - Leaderboard ranking
 12. Ride Sharing Apps Match drivers & riders, estimate fare, routes	- Graph (Shortest Path) - Heap (Nearest Driver) - Hashing (User / Location) - Greedy (Best Match)	Quick matching, shortest route and dynamic pricing calculation.	- Find nearest driver - Ride fare estimation - Optimized route suggestion

★ The right data structure + algorithm = Efficient solution + Happy users!

Top 10 Must Solve Real Life Problems

- Design a Search Autocomplete System
- Find Shortest Route in a City (Maps)
- Room Booking System (No Overlap)
- LRU Cache Implementation
- Log File Analysis (Most Frequent IP)
- Design a Parking Lot
- Stock Span Problem
- Design a Rate Limiter (API Requests)
- Social Network (Friend Suggestions)
- File Compression using Huffman Coding

Key Skills You Develop

- ✓ Choosing right data structure
- ✓ Optimizing time & space
- ✓ Handling real world constraints
- ✓ Writing clean & scalable code
- ✓ Thinking logically & systematically

Popular Real Life Problems - Quick Pointers

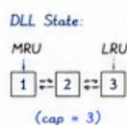
Problem	DSA Used	Key Idea / Approach	Time Complexity
Search Autocomplete	Trie	Store all words in Trie and fetch suggestions by prefix.	$O(m)$ (m = prefix length)
Shortest Path in Map	Dijkstra / BFS	Model map as graph, use shortest path algorithm.	$O((V + E) \log V)$
Room Booking	Sorting + Sweep Line	Sort by start time, use min-heap to track rooms.	$O(n \log n)$
LRU Cache	HashMap + DLL	HashMap for $O(1)$ access, DLL for recency.	$O(1)$
Log File Analysis	HashMap	Count frequency of each IP and find max.	$O(n)$
Parking Lot	Stack / Queue / Hashing	Track slots, vehicles and availability.	$O(1)$ average
Stock Span	Stack	Use decreasing stack to find previous greater.	$O(n)$
Rate Limiter	Sliding Window + Hashing	Track timestamps of requests in sliding window.	$O(1)$
Friend Suggestions	Graph (Mutual Friends)	Use graph intersections to find suggestions.	$O(d_1 + d_2)$
File Compression (Huffman)	Heap + Tree	Build Huffman Tree using min-heap.	$O(n \log n)$

Tips Box

- ✓ Understand the problem
- ✓ Identify entities
- ✓ Find relationships
- ✓ Choose the right DSA
- ✓ Code, test & optimize
- ✓ Handle edge cases
- ✓ Think about scalability

Example 1: LRU Cache ($O(1)$)

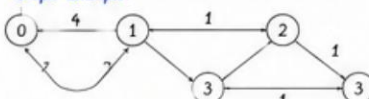
```
// Using HashMap + Doubly Linked List (DLL)
class LRUCache {
    int cap;
    unordered_map<int, list<pair<int,int>>::iterator> mp;
    list<pair<int,int>> dll; // {key, value}
public:
    LRUCache(int capacity) { cap = capacity; }
    int get(int key) {
        if(mp.find(key) == mp.end()) return -1;
        auto it = mp[key];
        int val = it->second;
        dll.erase(it);
        dll.push_front({key, val});
        mp[key] = dll.begin();
        return val;
    }
    void put(int key, int value) {
        if(mp.find(key) != mp.end()) {
            dll.erase(mp[key]);
            mp.erase(key);
        } else if(dll.size() == cap) {
            mp.erase(dll.back().first);
            dll.pop_back();
        }
        dll.push_front({key, value});
        mp[key] = dll.begin();
    }
};
```



Example 2: Shortest Path in Graph (Dijkstra)

```
// Min Heap (Priority Queue)
vector<int> dijkstra(int n, vector<vector<pair<int,int>>&g, int src){
    vector<int> dist(n, 1e9);
    priority_queue<pair<int,int>, 
        vector<pair<int,int>, greater<pair<int,int>>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while(!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if(d > dist[u]) continue;
        for(auto [v, w] : g[u]) {
            if(dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

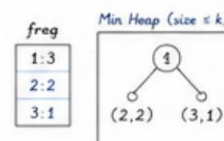
Graph Example:



Example 3: Top K Frequent Elements

```
// Using HashMap + Min Heap
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for(int x : nums) freq[x]++;
    priority_queue<pair<int,int>, 
        vector<pair<int,int>, greater<pair<int,int>>> pq;
    for(auto &p : freq) {
        pq.push({p.second, p.first});
        if(pq.size() > k) pq.pop();
    }
    vector<int> res;
    while(!pq.empty()) {
        res.push_back(pq.top().second);
        pq.pop();
    }
    reverse(res.begin(), res.end());
    return res;
}
```

Example:
nums = [1,1,1,2,2,3], k = 2
Output: [1, 2]



★ Final Thoughts

Practice solving real life problems daily.
Build, break and improve.
That's how you become interview ready!

Best Resources to Practice

- LeetCode (Top Interview Questions)
- HackerRank (DSA Practice)
- Codeforces (Problem Solving)
- GFG (Real Life Problem Lists)

★ Remember

DSA is a skill
Practice
Daily

You Got This!

Keep practicing.

DSA PLACEMENT QUESTIONS

Real Life Problem Solving with DSA (Continued)

★ Think : Model the real world → Choose the right DSA → Implement → Optimize → Test → Improve

More Real Life Scenarios

Scenario	DSA Used	Why It Works	Examples
 13. Movie Recommendation Recommend movies to users based on their taste	- Matrix (User-Movie) - HashMap (User Preferences) - Heap (Top Recommendations) - Graph (Similar Users/Items)	Efficient similarity computation, fast top-k recommendations and handling sparse data.	- Netflix recommendations - Trending now for you - Similar movies
 14. Food Delivery Apps Search nearby restaurants, delivery time, best offers	- Graph (Shortest Path) - Heap (Nearest Restaurant) - HashMap (Offers/Coupons) - Sorting (Filter & Sort)	Find nearest restaurants, optimize delivery routes, apply offers & sort results.	- Near me restaurants - Fast delivery - Best offers near you
 15. Flight / Train Booking Search routes, availability, prices, schedules	- Graph (Routes) - Priority Queue (Cheapest) - Binary Search (Availability) - HashMap (Schedules)	Efficient route search, cheapest option first, quick seat availability check.	- Cheapest flights - Alternative routes - Seat availability
 16. Version Control (Git) Track changes, branches, merges, commits	- Tree (Commit Tree) - Hashing (Commit IDs) - Graph (Branches & Merges) - Stack (Undo Operations)	Tree/graph represents history, hashing ensures uniqueness, stack helps with undo/redo.	- Git commit history - Branching & merging - Revert changes
 17. Search Autocomplete Show suggestions as user types in search box	- Trie (Prefix Search) - Heap (Top Suggestions) - HashMap (Search Frequency)	Trie allows fast prefix search, heap gets top suggestions, frequency improves ranking.	- Google search suggestions - YouTube autocomplete - App search suggestions
 18. Network Monitoring Monitor packets, detect anomalies, bandwidth usage	- HashMap (IP to Stats) - Sliding Window (Rate Limit) - Heap (Top Traffic) - Graph (Network Topology)	Efficient tracking, real time rate limiting, top usage alerts, and topology view.	- Top bandwidth users - Intrusion detection - Real time alerts

How to Convert Real Life Problem into DSA Solution?

- 1. Understand**
- Read problem carefully. Identify inputs, outputs & constraints.
- 2. Think & Model**
- Break into parts. Find entities, relationships, actions.
- 3. Choose DSA**
- Pick the best data structure(s) and algorithm(s) for the model.
- 4. Design**
- Write pseudo code or draw diagrams. Plan the approach.
- 5. Implement**
- Code the solution cleanly and modularly.
- 6. Optimize**
- Improve time/space complexity if needed.
- 7. Test**
- Test with edge cases and large inputs.
- 8. Refine & Learn**
- Review, refactor and learn from the solution.

Real Life Problem → DSA Mapping Examples

Real World Need	Best DSA / Approach
Find something fast	Hashing, Binary Search, Trie
Find shortest / quickest way	Graphs (BFS, Dijkstra), Heuristics
Handle many requests	Queues, Stacks, HashMap, Trees
Rank / Top K results	Heap / Priority Queue, Sorting
Store relationships	Graphs, HashMap
Handle time based data	Sliding Window, Queue, Stack
Undo / Redo operations	Stack
Count / Frequency	HashMap
Hierarchical data	Trees (Binary Tree, Trie)

Tips for Interviews

- ✓ Always clarify the problem with examples.
- ✓ Ask the interviewer if any constraints are more important.
- ✓ Start with brute force (if small input).
- ✓ Identify patterns early.
- ✓ Think out loud.
- ✓ Write clean, readable code.
- ✓ Test your code with edge cases.
- ✓ Analyze time & space complexity.

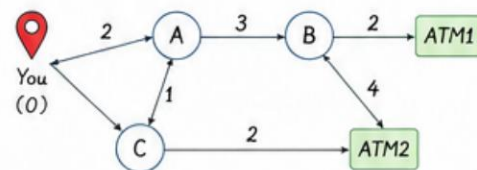
Real Life Problem Walkthrough

Problem: Find the nearest ATM from my current location.

Approach:

- Model as a graph: intersections as nodes, roads as edges with distances.
- Use Dijkstra's Algorithm to find the shortest distance from current location to all ATMs.
- Return the nearest ATM (minimum distance).

Example Diagram



Result:
 Nearest ATM
 = ATM1
 Distance = 5

Common Patterns in Real Life Problems

- | | |
|-------------------------------|-------------------------------------|
| • Search / Lookup | → Hashing, Binary Search, Trie |
| • Shortest Path / Route | → Graph, BFS, Dijkstra, A* |
| • Top K / Ranking | → Heap, Priority Queue, Sorting |
| • Scheduling / Timing | → Priority Queue, Interval Problems |
| • Counting / Frequency | → HashMap, Arrays |
| • Connections / Relationships | → Graph, Union Find |
| • Sequential Decision Making | → DP, Greedy, Backtracking |
| • Real Time Data | → Sliding Window, Queues |

Edge Cases to Always Check

- ✓ Empty input
- ✓ Single element
- ✓ All elements same
- ✓ Large input size
- ✓ Max/Min values
- ✓ No possible solution
- ✓ Multiple possible solutions

Remember!

- ★ DSA is a tool.
- ★ Problem solving is the skill.
- ★ Real world problems are diverse - keep practicing!
- ★ The more you practice, the better you get!



★ **Final Note:** Observe the world around you, pick a problem, model it using DSA